AmyE123 / SOLID-Principles-Study

Example of Single Responsibility Principle Refactors. #3

Open AmyE123 wants to merge 1 commit into `main` from `Single-Responsibility-Principle`

- Single-Responsibility-Principle
  - Level
    - ChargeSound.cs
    - HoldMouse.cs
    - MapFlipper.cs
  - Player
    - PlayerCollisionHandler.cs
    - PlayerDeath.cs
    - PlayerGridPreview.cs
    - PlayerManager.cs
    - PlayerMovement.cs
    - PlayerMoveOnRelease.cs
  - UI
    - UI_Ring.cs

Single-Responsibility-Principle/Level/ChargeSound.cs +24 ☐ Viewed

```diff
@@ -0,0 +1,24 @@
+using UnityEngine;
+
+// This file has been created, and it has a single responsibility to play a sound for each charge step.
+public class ChargeSound : MonoBehaviour
+{
+    public HoldMouse holdMouse;
+    public SoundFX soundFX;
+    public AudioClip[] gridSounds;
+
+    void OnEnable()
+    {
+        holdMouse.ChargeStepped += PlayStepSound;
+    }
+
+    void OnDisable()
+    {
+        holdMouse.ChargeStepped -= PlayStepSound;
+    }
+
+    void PlayStepSound(int step)
+    {
+        soundFX.PlaySound(gridSounds[step - 1]);
+    }
+}
```

Single-Responsibility-Principle/Level/HoldMouse.cs +20 -55 ☐ Viewed

```diff
@@ -1,42 +1,32 @@
-using System.Collections;
-using System.Collections.Generic;
+using System;
 using UnityEngine;
-using UnityEngine.UI;
-using TMPro;
-using DG.Tweening;
 
+// This file has been edited, to have a single responsibility. Beforehand it did a lot more, but now it only checks the mouse input for tap or hold, and it knows nothing about sound, ui, players or the map, other components would subscribe to its events.
 public class HoldMouse : MonoBehaviour
 {
-    [Header("References")]
-    public UI_Ring ring_UI;
-    public PlayerMovement[] players;
-    public LevelManager levelManager;
-    public SoundFX soundFX;
-    public AudioClip[] gridSounds;
-
-
     [Header("Data Values")]
-    [Space(20)]
     public int numberSpaces = 0;
     public float clickDelay = 0.25f;
-    bool isCharging = false;
-    public bool upright = true;
     public float fullChargeDelay = 0.5f;
 
+    const float ClickDelayReset = 0.25f;
+    const int MaxSpaces = 5;
 
-    void start()
-    {
+    bool isCharging = false;
 
-    }
+    public event Action<int> ChargeStepped;
+
+    public event Action<int> ChargeReleased;
+
+    public event Action Tapped;
 
     void Update()
     {
         ButtonPress();
 
-        if(numberSpaces >= 5)
+        if (numberSpaces >= MaxSpaces)
         {
             numberSpaces = 1;
-
         }
     }
 
@@ -49,63 +39,38 @@ void ButtonPress()
             {
                 OnHoldDown();
             }
-
         }
         else if (isCharging)
         {
-            MoveSpaces();
+            Release();
         }
         else
         {
             numberSpaces = 0;
         }
 
-        if(Input.GetMouseButtonDown(0))
-        {
-
-        }
         if (Input.GetMouseButtonUp(0))
         {
-            clickDelay = 0.25f;
+            clickDelay = ClickDelayReset;
             if (numberSpaces <= 0)
             {
-                levelManager.FlipMap(upright);
-                upright = !upright;
+                Tapped?.Invoke();
             }
         }
     }
 
-    void MoveSpaces()
-    {
-        ring_UI.FadeOut_Fin();
-        isCharging = false;
-        clickDelay = 0.25f;
-        //player.PlayerMove(numberSpaces);
-        foreach(PlayerMovement p in players)
-        {
-            p.PlayerMove(numberSpaces);
-        }
-    }
-
     void OnHoldDown()
     {
-        ring_UI.ChargeRing();
         isCharging = true;
         numberSpaces += 1;
-        clickDelay = 0.25f;
-        soundFX.PlaySound(gridSounds[numberSpaces - 1]);
+        clickDelay = ClickDelayReset;
+        ChargeStepped?.Invoke(numberSpaces);
     }
 
-    void OnClick()
-    {
-        if(Input.GetMouseButtonDown(0))
+    void Release()
     {
-
-        }
-        if(Input.GetMouseButtonUp(0))
-        {
-            clickDelay = 0.25f;
-        }
+        isCharging = false;
+        clickDelay = ClickDelayReset;
+        ChargeReleased?.Invoke(numberSpaces);
     }
-
 }
```

Single-Responsibility-Principle/Level/MapFlipper.cs +25 ☐ Viewed

```diff
@@ -0,0 +1,25 @@
+using UnityEngine;
+
+// This file has been created, and it has a single responsibility to flip the level map whenever the player taps.
+public class MapFlipper : MonoBehaviour
+{
+    public HoldMouse holdMouse;
+    public LevelManager levelManager;
+    public bool upright = true;
+
+    void OnEnable()
+    {
+        holdMouse.Tapped += Flip;
+    }
+
+    void OnDisable()
+    {
+        holdMouse.Tapped -= Flip;
+    }
+
+    void Flip()
+    {
+        levelManager.FlipMap(upright);
+        upright = !upright;
+    }
+}
```

Single-Responsibility-Principle/Player/PlayerCollisionHandler.cs +30 ☐ Viewed

```diff
@@ -0,0 +1,30 @@
+using UnityEngine;
+
+// This file has been created, and it has a single responsibility to deal with the collissions of the player.
+public class PlayerCollisionHandler : MonoBehaviour
+{
+    public PlayerMovement movement;
+    public PlayerDeath death;
+    public ButtonCommand button;
+    public Transform blockArea1;
+
+    public SoundFX soundFX;
+    public AudioClip buttonClick;
+
+    void OnTriggerEnter(Collider other)
+    {
+        if (other.CompareTag("enemy"))
+        {
+            death.Die();
+        }
+        else if (other.CompareTag("button"))
+        {
+            button.LowerBlockage();
+            soundFX.PlaySound(buttonClick);
+        }
+        else if (other.CompareTag("block"))
+        {
+            movement.StopAt(blockArea1.position);
+        }
+    }
+}
```

Single-Responsibility-Principle/Player/PlayerDeath.cs +25 ☐ Viewed

```diff
@@ -0,0 +1,25 @@
+using UnityEngine;
+
+// This file has been created, and it has a single responsibility to handle the player's death with the fx and respawn.
+public class PlayerDeath : MonoBehaviour
+{
+    public PlayerMovement movement;
+    public Transform player;
+    public ParticleSystem playerExplosion;
+    public Vector3 startPos;
+    public Vector3 deathPos;
+
+    public SoundFX soundFX;
+    public AudioClip deathSound;
+
+    public void Die()
+    {
+        soundFX.PlaySound(deathSound);
+        deathPos = movement.playerPos;
+
+        var explosion = Instantiate(playerExplosion, player);
+        explosion.transform.SetParent(null);
+
+        movement.ResetTo(startPos);
+    }
+}
```

Single-Responsibility-Principle/Player/PlayerGridPreview.cs +29 ☐ Viewed

```diff
@@ -0,0 +1,29 @@
+using UnityEngine;
+
+// This file has been created, and it has a single responsibility to deal with the player grids when they charge their movement
+public class PlayerGridPreview : MonoBehaviour
+{
+    public HoldMouse holdMouse;
+    public PlayerMovement movement;
+    public GameObject[] gridTiles;
+
+    void Start()
+    {
+        ShowTiles(0);
+    }
+
+    void Update()
+    {
+        int charged = holdMouse.numberSpaces;
+        bool inRange = charged >= 1 && charged <= gridTiles.Length;
+        ShowTiles(movement.isActive && inRange ? charged : 0);
+    }
+
+    void ShowTiles(int count)
+    {
+        for (int i = 0; i < gridTiles.Length; i++)
+        {
+            gridTiles[i].SetActive(i < count);
+        }
+    }
+}
```

Single-Responsibility-Principle/Player/PlayerManager.cs -27 ☐ Viewed

Load Diff

This file was deleted.

Single-Responsibility-Principle/Player/PlayerMovement.cs +14 -163 ☐ Viewed

```diff
@@ -1,184 +1,35 @@
-using System.Collections;
-using System.Collections.Generic;
 using UnityEngine;
 using DG.Tweening;
-using UnityEngine.ParticleSystemJobs;
 
+// This file has been edited, and it now has a single responsibility of moving the player along the grid, moving the input, grid stuff, death, collisions and sound to other files.
 public class PlayerMovement : MonoBehaviour
 {
     public Vector3 playerPos;
     public float speed = 0.5f;
-    public float moveDelay = 0.5f;
     public bool isActive = true;
-    public GameObject[] gridTiles;
-    public Transform player;
-    public ParticleSystem playerExplosion;
-    public Vector3 startPos;
-    public Vector3 deathPos;
-    public HoldMouse holdMouse;
-    float respawnDelay = 0.2f;
-    public ButtonCommand button;
+    public Vector3 moveDirection = Vector3.right;
 
-    public SoundFX soundFX;
-    public AudioClip buttonClick;
-    public AudioClip deathSound;
-
-    public Transform blockArea1;
-    public bool isHit;
-
-
-
-    void Start()
-    {
-        //startPos = player.position;
-        foreach(GameObject g in gridTiles)
-        {
-            g.SetActive(false);
-        }
-    }
-
-    void Update()
-    {
-        respawnDelay -= Time.deltaTime;
-        if(holdMouse.numberSpaces == 1 && isActive)
-        {
-            gridTiles[0].SetActive(true);
-            gridTiles[1].SetActive(false);
-            gridTiles[2].SetActive(false);
-            gridTiles[3].SetActive(false);
-
-        }
-        else if(holdMouse.numberSpaces == 2 && isActive)
-        {
-            gridTiles[0].SetActive(true);
-            gridTiles[1].SetActive(true);
-            gridTiles[2].SetActive(false);
-            gridTiles[3].SetActive(false);
-        }
-        else if(holdMouse.numberSpaces == 3 && isActive)
-        {
-            gridTiles[0].SetActive(true);
-            gridTiles[1].SetActive(true);
-            gridTiles[2].SetActive(true);
-            gridTiles[3].SetActive(false);
-        }
-        else if(holdMouse.numberSpaces == 4 && isActive)
-        {
-            gridTiles[0].SetActive(true);
-            gridTiles[1].SetActive(true);
-            gridTiles[2].SetActive(true);
-            gridTiles[3].SetActive(true);
-        }
-        else
-        {
-            gridTiles[0].SetActive(false);
-            gridTiles[1].SetActive(false);
-            gridTiles[2].SetActive(false);
-            gridTiles[3].SetActive(false);
-        }
-
-        // if(playerCol.tag == "enemy")
-        // {
-        //     isHit = true;
-        // }
-        if(isHit)
-        {
-            player.localPosition = startPos;
-            playerPos = startPos;
-            speed = 0.5f;
-            isHit = false;
-        }
-
-        // switch(holdMouse.numberSpaces)
-        // {
-        //     case 1:
-        //         gridTiles[0].SetActive(true);
-        //         gridTiles[1].SetActive(false);
-        //         gridTiles[2].SetActive(false);
-        //         gridTiles[3].SetActive(false);
-        //         break;
-        //     case 2:
-        //         gridTiles[0].SetActive(true);
-        //         gridTiles[1].SetActive(true);
-        //         gridTiles[2].SetActive(false);
-        //         gridTiles[3].SetActive(false);
-        //         break;
-        //     case 3:
-        //         gridTiles[0].SetActive(true);
-        //         gridTiles[1].SetActive(true);
-        //         gridTiles[2].SetActive(true);
-        //         gridTiles[3].SetActive(false);
-        //         break;
-        //     case 4:
-        //         gridTiles[0].SetActive(true);
-        //         gridTiles[1].SetActive(true);
-        //         gridTiles[2].SetActive(true);
-        //         gridTiles[3].SetActive(true);
-        //         break;
-        //     default:
-        //         gridTiles[0].SetActive(false);
-        //         gridTiles[1].SetActive(false);
-        //         gridTiles[2].SetActive(false);
-        //         gridTiles[3].SetActive(false);
-        //         break;
-        // }
-
-    }
+    const float DefaultSpeed = 0.5f;
 
     public void PlayerMove(int numberOfSpaces)
     {
-        // if(isActive)
-        // {
-        //     playerPos += Vector3.right * numberOfSpaces;
-        //     Debug.Log("right");
-        //     transform.DOMove(playerPos, speed);
-        // }
-        if(isActive && holdMouse.players[0])
-        {
-            playerPos += Vector3.right * numberOfSpaces;
-            transform.DOLocalMove(playerPos, speed);
-            //transform.localPosition = playerPos;
-        }
-        else if(isActive && holdMouse.players[1])
-        {
-            playerPos += Vector3.left * numberOfSpaces;
+        if (!isActive) return;
+
+        playerPos += moveDirection * numberOfSpaces;
         transform.DOLocalMove(playerPos, speed);
     }
-    }
 
-    void OnTriggerEnter(Collider other)
-    {
-        Debug.Log("trigger");
-        if(other.tag == "enemy")
-        {
-            isHit = true;
-            DOTween.Kill(transform);
-            soundFX.PlaySound(deathSound);
-            Debug.Log("death");
-            deathPos = playerPos;
-            var deathPosition = Instantiate(playerExplosion, player);
-            deathPosition.transform.SetParent(null);
-            speed = 0;
-            playerTransform();
-        }
-        else if(other.tag == "button")
-        {
-            button.LowerBlockage();
-            soundFX.PlaySound(buttonClick);
-        }
-        else if(other.tag == "block")
+    public void StopAt(Vector3 position)
     {
-            //playerPos -= Vector3.right;
-            transform.DOLocalMove(blockArea1.position, speed);
-            //transform.position = endblock.position;
-            playerPos = blockArea1.position;
-        }
+        transform.DOLocalMove(position, speed);
+        playerPos = position;
     }
 
-    void playerTransform()
+    public void ResetTo(Vector3 position)
     {
-        player.localPosition = startPos;
-        playerPos = startPos;
-        speed = 0.5f;
+        DOTween.Kill(transform);
+        transform.localPosition = position;
+        playerPos = position;
+        speed = DefaultSpeed;
     }
 }
```

Single-Responsibility-Principle/Player/PlayerMoveOnRelease.cs +26 ☐ Viewed

```diff
@@ -0,0 +1,26 @@
+using UnityEngine;
+
+// This file has been created to have a single responsibility of making the movement on hold down charge work and move the player
+public class PlayerMoveOnRelease : MonoBehaviour
+{
+    public HoldMouse holdMouse;
+    public PlayerMovement[] players;
+
+    void OnEnable()
+    {
+        holdMouse.ChargeReleased += MovePlayers;
+    }
+
+    void OnDisable()
+    {
+        holdMouse.ChargeReleased -= MovePlayers;
+    }
+
+    void MovePlayers(int spaces)
+    {
+        foreach (PlayerMovement p in players)
+        {
+            p.PlayerMove(spaces);
+        }
+    }
+}
```

Single-Responsibility-Principle/UI/UI_Ring.cs +24 -7 ☐ Viewed

```diff
@@ -1,10 +1,10 @@
-using System.Collections;
-using System.Collections.Generic;
 using UnityEngine;
 using UnityEngine.UI;
 using TMPro;
 using DG.Tweening;
 
+// This file has been edited, and it now has the single responsibility of displaying the hold down charge ring at the cursor point.
+// Should rename to UIRing.cs to fit other naming conventions
 public class UI_Ring : MonoBehaviour
 {
     [Header("References")]
@@ -14,25 +14,44 @@ public class UI_Ring : MonoBehaviour
     [Space(20)]
     float FillMultiplier = 0.25f;
 
-
     [Header("UI References")]
     [Space(20)]
     public Transform circlePos;
     public Image fillRing;
     public Image outerRing;
     public TextMeshProUGUI numberText;
 
+    void OnEnable()
+    {
+        holdManager.ChargeStepped += OnChargeStepped;
+        holdManager.ChargeReleased += OnChargeReleased;
+    }
+
+    void OnDisable()
+    {
+        holdManager.ChargeStepped -= OnChargeStepped;
+        holdManager.ChargeReleased -= OnChargeReleased;
+    }
+
     void Start()
     {
         fillRing.fillAmount = 0;
     }
 
     void Update()
     {
-        var mousePos = Input.mousePosition;
-        circlePos.position = new Vector3(mousePos.x, mousePos.y, mousePos.z);
+        circlePos.position = Input.mousePosition;
         numberText.SetText(holdManager.numberSpaces.ToString());
+    }
+
+    void OnChargeStepped(int step)
+    {
+        ChargeRing();
+    }
 
+    void OnChargeReleased(int spaces)
+    {
+        FadeOut_Fin();
     }
 
     public void FadeOut_Fin()
@@ -45,11 +64,9 @@ public void FadeOut_Fin()
 
     public void ChargeRing()
     {
-        Debug.Log(holdManager.numberSpaces/4.0f);
         fillRing.DOFade(0.8f, 1);
         outerRing.DOFade(0.5f, 1);
         numberText.DOFade(1, 1);
         fillRing.fillAmount += FillMultiplier;
     }
-
 }
```